The actual validation happens in the valid? method. Validation#save_with_validation returns false if the validation fails, or if the caller explicitly disables validation. Otherwise, it just calls the original save_without_validation. Validation#save_with_validation! raises an exception if the validation fails, and otherwise falls back to the original save_with_validation!.

This is how alias_method_chain was used around the times of Rails 2. Things have changed since then, as I will explain next.

The Fall of alias_method_chain

In the previous two chapters, you've seen that the libraries in Rails are mostly built by assembling modules. Back in Rails 2, many of those modules used alias_method_chain to wrap functionality around the methods of their includers. The authors of libraries that extended Rails adopted the same mechanism to wrap their own functionality around the Rails methods. As a result, alias_method_chain was used all over the place, both in Rails and in dozens of third-party libraries.

alias_method_chain was good at removing duplicated aliases, but it also came with a few problems of its own. For a start, alias_method_chain is just an encapsulation of an *Around Alias* *(134)*, and Around Aliases have the subtle problems that you might remember from *The Thor Example*, on page 133. To make things worse, alias_method_chain turned out to be too clever for its own good: with all the method renaming and shuffling that was going on in Rails, it could become hard to track which version of a method you were actually calling.

However, the most damning issue of alias_method_chain was that it was simply unnecessary in most cases. Ruby is an object-oriented language, so it provides a more elegant, built-in way of wrapping functionality around an existing method. Think back to our example of adding enthusiasm to the greet method:

part2/greet_with_super.rb

```
module Greetings
  def greet
    "hello"
  end
end

class MyClass
  include Greetings
end

MyClass.new.greet  # => "hello"
```